Multiplication and Division

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

#B

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Fall 2020

Class announcements

Multiplication by repeated addition

- in A × B, the **product** (answer) is “B copies of A, added together"

3 × 6 = 18

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18

how many additions would it take to calculate

2 x 500,000,000?

Back to grade school

- remember your multiplication tables?
- binary is *so much easier*
- if we list 0 too, the product logic looks awfully familiar…

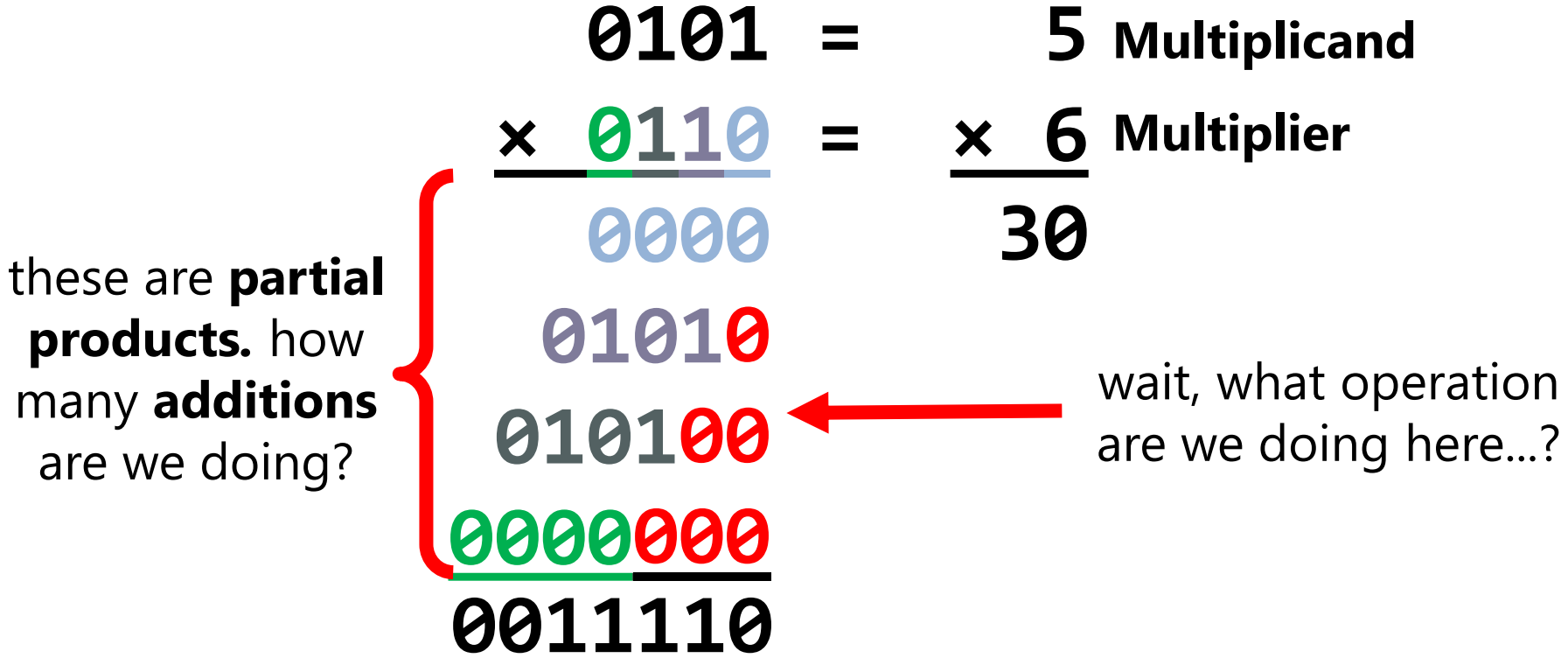
×	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

×	1
1	1

A	B	P
0	0	0
0	1	0
1	0	0
1	1	1

Just like you remember

- you know how to multiply, riiiight?



Wait, why does this work?

- what are we *actually doing* with this technique?
- remember how positional numbers are really polynomials?

FOIL...

$$78 \times 54 = 70 \times 50 + 70 \times 4 + 8 \times 50 + 8 \times 4$$

we're eliminating many addition steps by **grouping them together.**

$$= 78 \times 50 + 78 \times 4$$

we group them together by **powers of the base.**

How many bits?

- when we *added* two n-digit/bit numbers, how many digits/bits was the sum?
- how about for multiplication?
- when you **multiply** two n-digit/bit numbers, the product will be at most **2n** digits/bits
- so if we multiply two 32-bit numbers…
  - we could get a **64-bit result!** AAAA!
  - if we just ignored those extra 32 bits, or crashed, we'd be losing a lot of info.
  - so we have to **store it.**

```
   99
 × 99
 ----
 9801
```

```
  9999
 × 9999
 -------
 99980001
```

```
   1111
 × 1111
 -------
 11100001
```

How (and why) MIPS does it

- MIPS has **two more 32-bit registers, HI** and **LO.** if you do this:

```
mult t0, a0
```

- then HI = **upper 32 bits** of the product and LO = **lower 32 bits**
- to actually get the product, we use these:

```
mfhi t0 # move From HI (t0 = HI)
mflo t1 # move From LO (t1 = LO)
```

- the **mul** pseudo-op does a **mult** followed by an **mflo**
- MIPS does this for 2 reasons:
  - multiplication can *take longer than addition*
  - we'd otherwise have to *change two different registers at once*
- if you wanted to check for 32-bit multiplication overflow, how could you do it?

Signed multiplication

Grade school (but like, 6th, instead of 3rd)

- if you multiply two **signed** numbers, what's the rule?

Product

A	B	P
3	5	15
3	-5	-15
-3	5	-15
-3	-5	15

Sign

A	B	S
+	+	+
+	-	-
-	+	-
-	-	+

if the signs of the operands **differ**, the output is **negative**.

Don't repeat yourself

- we already have an algorithm to multiply **unsigned** numbers
- multiplying signed numbers is exactly the same (except for the signs)
- so why not use what we already made?

```
long prod = unsigned_mult(abs(A), abs(B));
if(sgn(A) == sgn(B))
    return prod;
else
    return -prod;
```

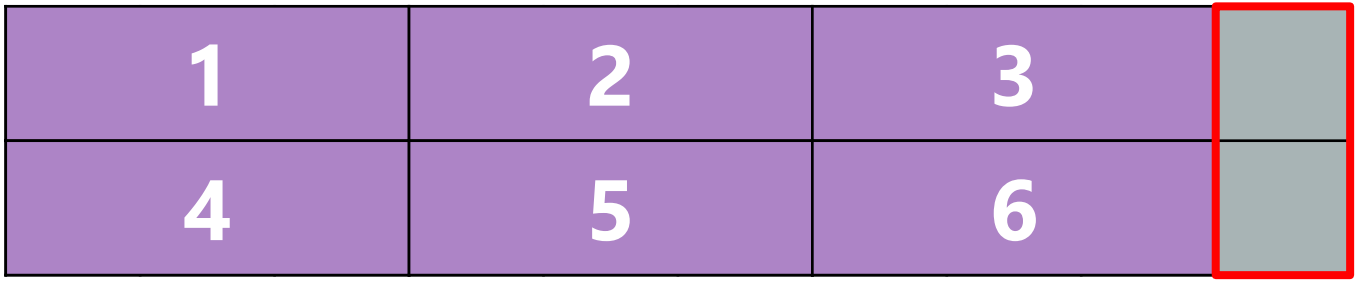
Division

Like multiplication, except… not really

If multiplication is repeated addition...

- ...is division *repeated subtraction?*
  - **yes.** it is.
- in A ÷ B, the **quotient** (answer) is "how many times can you subtract B from A until you can't anymore?"

20 ÷ 3 = 6 R 2



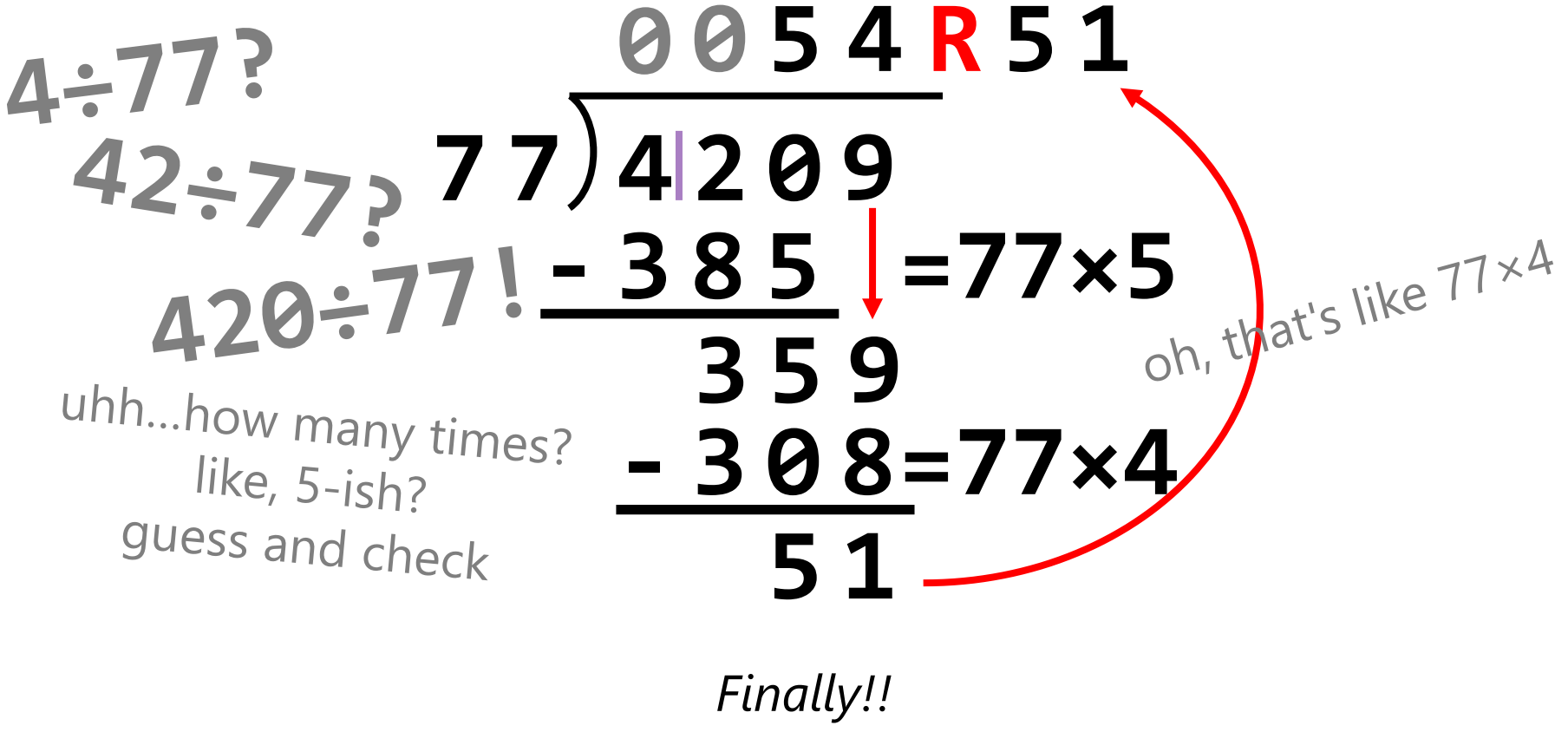
what about these lil guys?

how many subtractions would it take to calculate

1,000,000,000 ÷ 2?

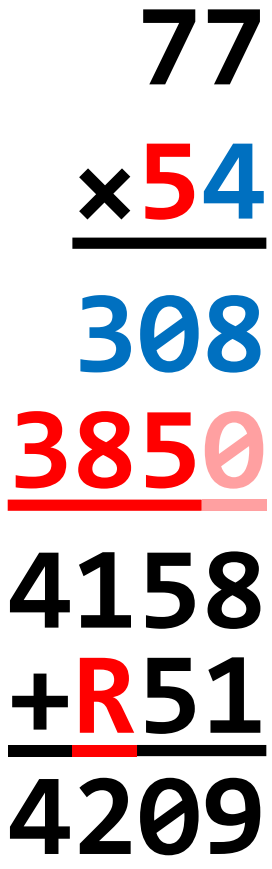
That's not what you learned in school, was it

- You learned something a *tiiiiiny bit* more complicated



What's going on?

- division is multiplication *backwards*. it's like ctrl+z.



we're finding **the partial products that add up to a total product**

division goes left-to-right because we find the partial products **from biggest to smallest**

and to make it *even more interesting*, there might be **a remainder**

and then there's division by 0.

multiplication is multiplying polynomials.
division is ***factoring polynomials.***

Another way of looking at it (animated)

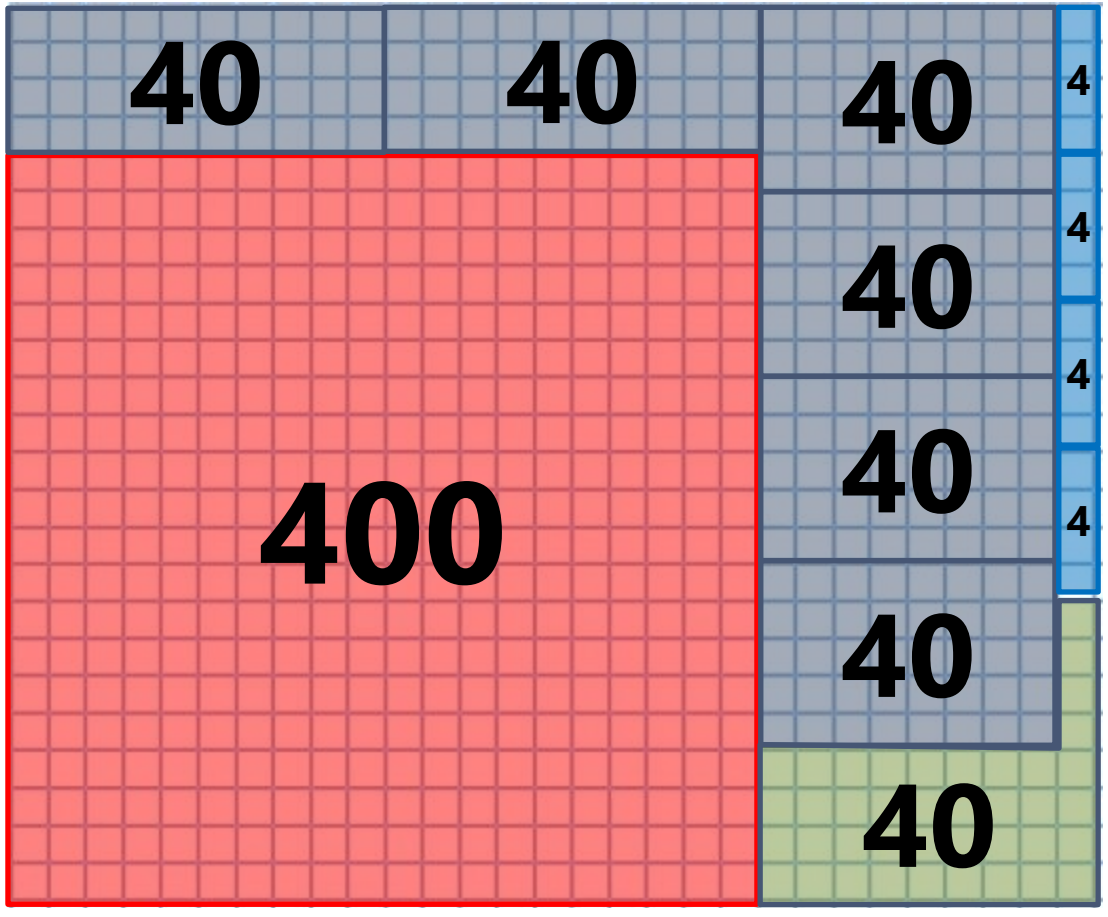
- let's say we want to do... (calculating random number)... **696÷4**

first we ask **how many 400s fit** **1**

then **how many 40s** **7**

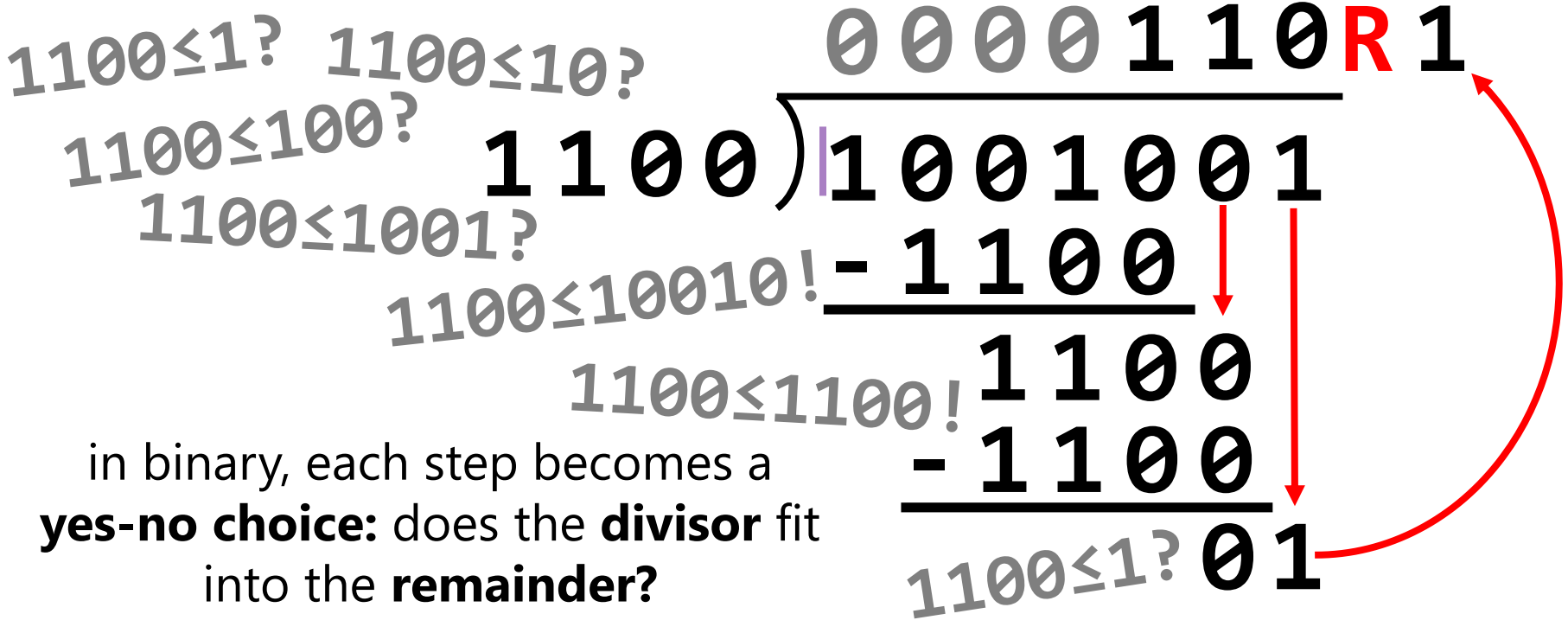
then **how many 4s** **4**

so we're saving time by doing groups of subtractions from biggest to smallest



Thanks, tiny multiplication table. Thable.

- at least multiplication in binary is easy, which simplifies division



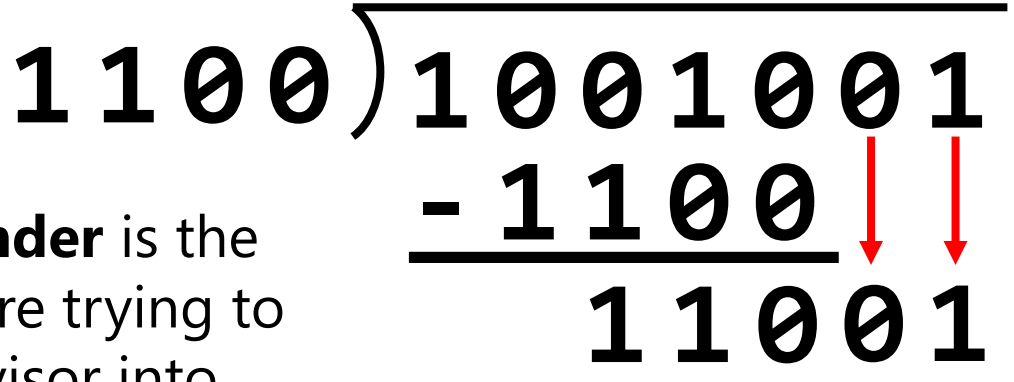
in binary, each step becomes a **yes-no choice:** does the **divisor** fit into the **remainder?**

Divisor? Dividend? Remainder?

the **divisor** divides the dividend

the **remainder** is the number we're trying to fit the divisor into

it starts off as the dividend…



the **dividend** is the number that is **being divided**

this is the new remainder.

but really, when we subtract something, we are **making the remainder smaller.**

Finding partial products, biggest to smallest (animated)

essentially we're starting with the divisor **shifted all the way left,** and **sliding it right**

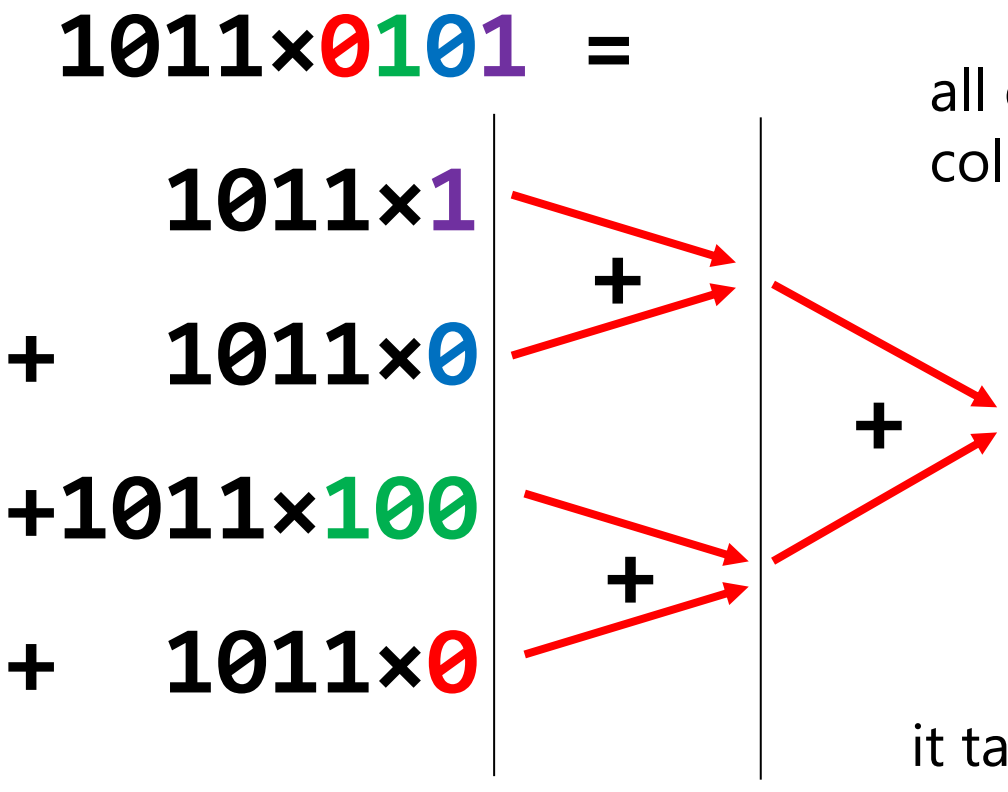
```
     0000110R1
   ┌──────────
1100)      1
   1100000000
```

so let's ALGORITHM-IZE IT

A few more odds and ends

Divide-and-conquer… in parallel

- an $n \times n$ digit multiplication **can be broken into n, $n \times 1$ digit ones**
- the partial products can be summed **in any order** (thanks, commutativity)



all operations in the same column can be done **in parallel.**

now our multiplication takes only **3 steps** instead of 4.

but this is a **O(log(n))** algorithm! so for 32 bits…

it takes **6 steps instead of 32!**

But division…

- if we try to do something similar, well…

1011÷101 =

1011÷101000 = 0

1011÷10100 = 0

1011÷1010 = 1 R 1

1011÷101 = 1 R 110??

what's the difference between **addition and subtraction?**

subtraction is *not* commutative.

you can do the steps in *any order*… but you can't do them *at the same time*.

we *cannot know the answer to this step*…

…until we know the answer to the *previous one*.

Division is fundamentally slower

- **each step depends on the previous one.**
- we *cannot* split it up into subproblems like with multiplication.
- the only way to make division faster is to **guess.**
  - SRT Division is a way of predicting quotient bits based on the next few bits of the dividend and divisor
  - but it can make mistakes and they have to be corrected
  - the original Pentium CPU in 1994 **messed this up**
    - and Intel pretended everything was OK
    - and people got mad
    - and they had to recall millions of them
    - and Intel lost half a billion dollars
      - lol

Doing modulo with AND

- in decimal, dividing by powers of 10 is trivial.

53884 ÷ 1000 = 53 R 884

- in binary, we can divide by powers of 2 easily with shifting
- and we can **get the remainder by masking!**

10010110 ÷ 1000 = 10010 R 110
10010110 >> 11 = 10010
10010110 & 0111 = 110

more generally: **a AND (2^n-1) = a % 2^n**

more generally: **a AND ((1<<n)-1) = a % 2^n**

Signed division

All roads lead to Rome… er, the Dividend

- how did we extend our multiplication algorithm to signed numbers?
- but how exactly do the rules work when you have **two results?**

the four values are related as:

Dividend = (Divisor × Quotient) + Remainder

If you do…	Java says…	Python says…
7 / 2	3 R 1	3 R 1
7 / -2	-3 R 1	-4 R -1
-7 / 2	-3 R -1	-4 R 1
-7 / -2	3 R -1	3 R -1

mathematicians would expect the remainder to **always be positive,** so the last row would be **4 R 1!**

check out https://en.wikipedia.org/wiki/Modulo_operation for this travesty

In Java -7/2 = -(7/2)
In Python -7/2 ≠ -(7/2)

Whaaaaaaaaaaaaaaaat

- no, really, it's not well-defined. there's no "right" answer.
- **watch out for this.**
  - I think I ran into it once because I was doing maths with angles in the range [-pi, pi)
  - most of the time, when you're dealing with modulo, you're dealing with positive values
  - Most languages I had used did **(-7 / 2)** as **-(7 / 2)**
    - this is "truncated division" (rounds towards 0)
  - but Python is gaining popularity and can sometimes be confusing
    - it uses "flooring division" (rounds towards -∞)
- so which does **arithmetic right shift** do?
  - it does **flooring division.**